

Discriminative Parameter Splitting for Large Language Models: Zero-Drift Network Expansion and Calculated Noise Symmetry Breaking

Anonymous ACL submission

Abstract

Pre-training Large Language Models (LLMs) from scratch is compute-intensive, and a deployed model whose capacity saturates on a downstream task cannot easily be enlarged without discarding its checkpoint. We present **Discriminative Parameter Splitting for LLMs (DPS-LLM)**, a progressive-growth method that doubles the intermediate width of every SwiGLU feed-forward block of a pre-trained Transformer while leaving its function unchanged at the moment of expansion. DPS-LLM combines (i) a *function-preserving* split—row duplication of the gate and up projections and column duplication of the down projection scaled by $\frac{1}{2}$ —for which we prove exact output invariance, and (ii) *Calculated Anti-Symmetric Noise*, a polarity-aware perturbation adapted from discriminative splitting of LSTM acoustic models, which breaks the gradient symmetry of duplicated neurons while cancelling first-order output drift. We evaluate on two open-weight model families, **Qwen2.5-0.5B** (494M→808M parameters) and **SmolLM-360M** (362M→598M), fine-tuned on GSM8K and Alpaca-Cleaned. Across all four model–dataset pairs the split models start at the perplexity of the base model (differences ≤ 0.02 , attributable to bfloat16 rounding), DPS-LLM reduces validation perplexity more than exact duplication in every case, and on the capacity-limited SmolLM-360M the relative perplexity reduction is $2.0\times$ (GSM8K) and $3.2\times$ (Alpaca) that of the unexpanded model. Code and notebooks are released to support reproduction.

1 Introduction

Transformer LLMs (Vaswani et al., 2017; Touvron et al., 2023; Yang et al., 2024) are almost always trained under a *static* paradigm: every dimension of the network is fixed *a priori*, the weights are drawn from a random initialization (Glorot and Bengio, 2010; He et al., 2015), and the model is

trained end-to-end. Scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) make the cost of this paradigm explicit: a larger model must be trained from scratch, spending much of its early budget on features a smaller model already possesses. The same rigidity appears at fine-tuning time. When a compact model saturates its parametric capacity on a domain-specific task, practitioners must either accept the plateau or restart with a larger architecture.

Progressive growth offers an alternative: begin with a compact *seed* model and enlarge it during training (Chen et al., 2016a; Gong et al., 2019; Chen et al., 2022; Wang et al., 2023). Two requirements make growth practical for modern LLMs. First, the expansion must be *function-preserving*, so that the grown model neither loses the seed’s knowledge nor suffers a loss spike. Second, the duplicated parameters must be *distinguishable*; otherwise identical copies receive identical gradients and the added capacity is never used (Chen et al., 2016a). Existing work on Transformers has addressed these requirements for pre-LayerNorm blocks with ReLU or GELU feed-forward layers (Chen et al., 2022; Wang et al., 2024), but gated SwiGLU blocks (Shazeer, 2020), which are now standard in LLaMA, Mistral, Gemma, Qwen and SmolLM, multiply two projections element-wise, so naive duplication doubles the block’s output.

In this paper we adapt *Discriminative Parameter Splitting* (Irfan, 2021), originally developed for growing LSTM acoustic models, to SwiGLU Transformers. Our contributions are:

- **An exact function-preserving SwiGLU split.** Duplicating the rows of W_{gate} and W_{up} and the columns of $\frac{1}{2}W_{down}$ doubles d_{ffn} with provably identical output (Theorem 1).
- **Calculated Anti-Symmetric Noise.** A perturbation that is signed by each weight’s own polarity and applied with opposite signs to

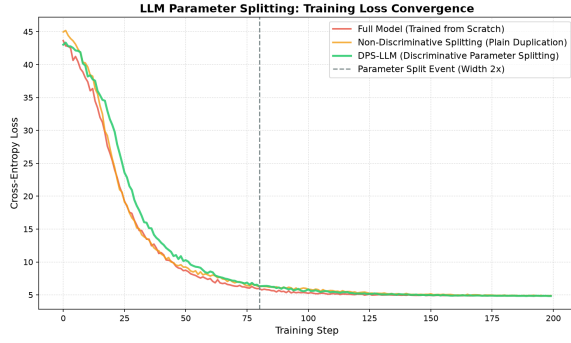


Figure 1: **Controlled split on a synthetic language-modelling task.** A two-layer SwiGLU Transformer with $d_{ffn} = 64$ is trained for 80 steps and then split to $d_{ffn} = 128$ (dashed line). Both split variants continue without a loss spike, whereas the $d_{ffn} = 128$ model trained from scratch spends its first 80 steps re-learning what the seed already knows. DPS-LLM and plain duplication converge to the same loss as the full model by step 200.

the two copies. It breaks gradient symmetry, cancels first-order output drift (Proposition 2), and preserves the sign of every weight except a negligible near-zero fraction (Proposition 1).

- **A controlled benchmark on real pre-trained LLMs.** On Qwen2.5-0.5B and SmolLM-360M, fine-tuned on GSM8K (Cobbe et al., 2021) and Alpaca-Cleaned (Taori et al., 2023), we compare the base model, exact duplication and DPS-LLM under identical budgets, and report both the successes and the regime in which the unexpanded model still wins.

We also describe, and release code for, two further regimes of the same operator—dense-to-Mixture-of-Experts (MoE) upcycling and LoRA rank doubling—which we leave for empirical evaluation in future work (Appendix A).

2 Related Work

Function-preserving growth. Net2Net (Chen et al., 2016a) introduced *Net2WiderNet* and *Net2DeeperNet*, which widen or deepen a network without changing its function, and noted that a small amount of noise is needed to break the symmetry of copied units. Network morphism (Wei et al., 2016) generalised the idea to arbitrary non-linear morphisms. For Transformers, progressive stacking (Gong et al., 2019) and staged training (Shen et al., 2022) grow depth and width of BERT-

style encoders; bert2BERT (Chen et al., 2022) extends Net2Net to attention blocks with function-preserving and advanced knowledge initialisation; LiGO (Wang et al., 2023) learns a linear growth operator from the small model; MSG (Yao et al., 2024) and LEMON (Wang et al., 2024) obtain lossless expansion for pre-LayerNorm blocks by masking or by an explicit average-and-cancel construction. Depth up-scaling (Kim et al., 2024) and HyperCloning (Samragh et al., 2024) apply the same principle to decoder-only LLMs, the latter by tiling weights of a small model into a wider one. DPS-LLM belongs to this family but targets the gated SwiGLU block specifically, where the $\frac{1}{2}$ factor must be placed on the down projection to preserve function.

Neuron splitting. Splitting steepest descent (Liu et al., 2019), Firefly descent (Wu et al., 2020) and GradMax (Evci et al., 2022) choose *which* neurons to split and *how* to perturb the copies using second-order or gradient information. These methods are principled but require extra computation per split; DPS-LLM instead splits every neuron and uses a closed-form, gradient-free perturbation whose only statistic is the standard deviation of the weight matrix.

Mixture-of-Experts upcycling. Sparse upcycling (Komatsuzaki et al., 2023) initialises MoE experts (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022) by cloning a dense MLP, and He et al. (2024) study this recipe at scale; Mixtral (Jiang et al., 2024) is a widely used dense-to-sparse architecture. Cloned experts are initially identical, so the same symmetry problem arises; our anti-symmetric noise is directly applicable (Appendix A).

Adapter rank growth. LoRA (Hu et al., 2022) fixes the adapter rank in advance; AdaLoRA (Zhang et al., 2023) and DyLoRA (Valipour et al., 2023) adapt the rank during training. Rank doubling by parameter splitting (Appendix A) is a complementary, function-preserving way to increase adapter capacity.

Discriminative splitting in speech. Irfan (2021) grew LSTM-RNN acoustic models by duplicating rows and columns of weight matrices and injecting a polarity-preserving perturbation, reporting faster convergence, less over-training and a 1.2% absolute word-error-rate reduction over Xavier initialisation on a Kaldi/VoxForge recipe. This paper transfers

that recipe to Transformers and adds the invariance scaling and the anti-symmetric pairing.

3 Method

3.1 Function-preserving SwiGLU width splitting

Modern decoder blocks (Shazeer, 2020; Touvron et al., 2023; Yang et al., 2024) use a gated feed-forward network

$$\text{FFN}(x) = W_{\text{down}} (\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x), \quad (1)$$

with $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d_{\text{ffn}} \times d_{\text{model}}}$, $W_{\text{down}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ffn}}}$, and $\text{SiLU}(z) = z \sigma(z)$ (Hendrycks and Gimpel, 2016; Elfving et al., 2018; Ramachandran et al., 2017). Let $h(x) = \text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x \in \mathbb{R}^{d_{\text{ffn}}}$.

To grow $d_{\text{ffn}} \rightarrow 2d_{\text{ffn}}$ we apply *rows extension* to the two incoming projections and *columns extension* with a $\frac{1}{2}$ factor to the outgoing projection:

$$W'_{\text{gate}} = \begin{bmatrix} W_{\text{gate}} + \Delta_{\text{gate}}^{(1)} \\ W_{\text{gate}} + \Delta_{\text{gate}}^{(2)} \end{bmatrix}, \quad (2)$$

$$W'_{\text{up}} = \begin{bmatrix} W_{\text{up}} + \Delta_{\text{up}}^{(1)} \\ W_{\text{up}} + \Delta_{\text{up}}^{(2)} \end{bmatrix}, \quad (3)$$

$$W'_{\text{down}} = \frac{1}{2} \begin{bmatrix} W_{\text{down}} & W_{\text{down}} \end{bmatrix} + \begin{bmatrix} \Delta_{\text{down}}^{(1)} & \Delta_{\text{down}}^{(2)} \end{bmatrix}. \quad (4)$$

Theorem 1 (Functional invariance). *With $\Delta = 0$, $\text{FFN}'(x) = \text{FFN}(x)$ for every input x .*

Proof. The stacked gate and up projections produce $[h(x); h(x)]$, so $\text{FFN}'(x) = \frac{1}{2}W_{\text{down}}h(x) + \frac{1}{2}W_{\text{down}}h(x) = W_{\text{down}}h(x)$. $\square$

Because every block is invariant and the residual stream, attention and normalisation layers are untouched, the whole model is invariant; the loss on any batch is unchanged in exact arithmetic. Without the $\frac{1}{2}$ factor the block’s output would double, which is the loss spike observed when SwiGLU networks are naively widened.

3.2 Calculated Anti-Symmetric Noise

If $\Delta = 0$, the two copies of every neuron are identical and receive identical gradients for the rest of training, so the expanded model has no more effective capacity than the seed (Chen et al., 2016a). Isotropic Gaussian noise breaks the tie but is sign-agnostic: it flips the polarity of small weights, which Irfan (2021) found harmful for pre-trained

LSTMs. We instead use a perturbation whose sign is dictated by the weight it perturbs. For a weight matrix W with element-wise standard deviation σ_W , draw $u \sim \mathcal{U}(0, 1)$ and $n \sim \mathcal{N}(0, 1)$ independently per element and set

$$f(W) = \left| \left(u - \frac{1}{2}\right) \cdot C \sigma_W \cdot n \right|, \quad (5)$$

$$\delta(W) = \text{sign}(W) \odot f(W), \quad (6)$$

$$\Delta^{(1)} = +\delta(W), \quad \Delta^{(2)} = -\delta(W). \quad (7)$$

The modulation constant $C = 10^{-4}$ is inherited from the original speech recipe; (5)–(6) are its Equations 5.7–5.8 with the noise scaled by σ_W so that the same C transfers across layers of different magnitude. Equation (7) is new: the two copies receive *opposite* perturbations. For W_{down} the noise is computed on the already-halved matrix.

Proposition 1 (Sign preservation). *Copy 1 satisfies $|w + \delta(w)| = |w| + f(w) > |w|$ for every weight, so no sign is flipped. Copy 2 satisfies $|w - \delta(w)| = ||w| - f(w)|$ and flips sign only if $|w| < f(w)$.*

Since $f(w) \leq \frac{1}{2}C\sigma_W|n|$, a flip requires $|w| < \frac{1}{2} \cdot 10^{-4}\sigma_W|n|$; for approximately Gaussian weights this is a fraction on the order of 10^{-4} of the matrix, and these weights carry negligible signal. In our runs the two copies therefore keep the pre-trained polarity of all but a vanishing set of parameters.

Proposition 2 (First-order drift cancellation). *With the pairing in (7), $\text{FFN}'(x) = \text{FFN}(x) + O(\|\delta\|^2)$.*

Proof. Write $h_1(x)$ and $h_2(x)$ for the hidden activations of the two copies. A first-order expansion in δ gives $h_1 = h + J\delta x + O(\delta^2)$ and $h_2 = h - J\delta x + O(\delta^2)$ for the same Jacobian J , so $\frac{1}{2}W_{\text{down}}(h_1 + h_2) = W_{\text{down}}h + O(\delta^2)$. The down-projection perturbation contributes $\delta_{\text{down}}h_1 - \delta_{\text{down}}h_2 = \delta_{\text{down}}(h_1 - h_2) = O(\delta^2)$. $\square$

The perturbation therefore does two things at once: it leaves the output unchanged to first order, and it makes $\nabla_{W^{(1)}}\mathcal{L} \neq \nabla_{W^{(2)}}\mathcal{L}$ from the very first update, because the two copies now see different pre-activations. Empirically the resulting perplexity difference at the split is at most 0.02 (Table 1) and is dominated by bfloat16 rounding rather than by the noise.

3.3 Applying the operator to a pre-trained LLM

The split is applied in place to every decoder layer of a Hugging Face checkpoint (Wolf et al., 2020):

the three linear layers are replaced by their expanded versions and `intermediate_size` is doubled in the configuration. No other tensors change. For Qwen2.5-0.5B this grows d_{ffn} from 4864 to 9728 in all 24 layers (+313.8M parameters); for SmolLM-360M from 2560 to 5120 in all 32 layers (+236.0M).

4 Experiments

4.1 Setup

Models. We use two instruction-tuned open-weight checkpoints that fit on a single 16 GB GPU after splitting: Qwen/Qwen2.5-0.5B-Instruct (Yang et al., 2024) (24 layers, $d_{model} = 896$, $d_{ffn} = 4864$) and HuggingFaceTB/SmolLM-360M-Instruct (Allal et al., 2024, 2025) (32 layers, $d_{model} = 960$, $d_{ffn} = 2560$). Both use SwiGLU blocks of the form in (1).

Datasets. GSM8K (Cobbe et al., 2021) (grade-school maths word problems with multi-step solutions) and Alpaca-Cleaned (Taori et al., 2023; yahma, 2023) (general instruction following). For each dataset we take the first 40 examples of the public training split, render them with a fixed template (Appendix B), and use examples 1–30 for fine-tuning and 31–40 as a held-out validation set that is never used for updates.

Growth conditions. (1) *Base*: the unmodified checkpoint. (2) *Naive split*: Eqs. (2)–(4) with $\Delta = 0$, i.e. exact duplication with the $\frac{1}{2}$ scaling. (3) *DPS-LLM*: the same split with Calculated Anti-Symmetric Noise, $C = 10^{-4}$.

Fine-tuning protocol. Every condition is fine-tuned with the same recipe (Table 2). Only the SwiGLU parameters (W_{gate} , W_{up} , W_{down} of every layer) are updated; attention, embeddings and the output head are frozen so that the comparison isolates the expanded block. Sequences are truncated to 128 tokens and the loss is the standard causal language-modelling loss over all tokens. Validation perplexity (Jelinek et al., 1977) is exp of the mean per-sequence cross-entropy over the 10 held-out sequences, measured before the first update (*Init PPL*) and after the last (*Final PPL*).

4.2 Results

Table 1 reports all twelve runs; Figure 3 visualises the relative perplexity reductions and the margin

of DPS-LLM over naive duplication, and Figure 2 shows the per-step training loss on Qwen2.5-0.5B.

Functional invariance holds on real checkpoints.

Immediately after splitting, validation perplexity equals that of the base model in every setting: $1.91 \rightarrow 1.91$ and $6.31 \rightarrow 6.31$ on GSM8K, $7.29 \rightarrow 7.28$ and $8.88 \rightarrow 8.86$ on Alpaca. The residual differences of at most 0.02 appear identically for naive duplication and DPS-LLM, so they stem from re-associating bfloat16 sums over a hidden dimension twice as wide, not from the noise. This confirms Theorem 1 and Proposition 2 at the scale of a full LLM.

Calculated noise beats exact duplication in all four settings.

DPS-LLM achieves a larger relative perplexity reduction than the naive split in every model–dataset pair, by +0.26, +0.07, +0.09 and +0.43 percentage points. The margins are small but consistent in sign, and they are largest on SmolLM/Alpaca, the setting with the least headroom in the base model. Under exact duplication the two copies of each neuron remain identical throughout fine-tuning, so the naive model is effectively the base model with a different numerical path; the margin therefore measures how much of the added capacity was actually used within the 25-step budget.

Expansion helps most when the base model is capacity-limited.

On SmolLM-360M the base model barely moves (1.03% on GSM8K, 0.71% on Alpaca), whereas DPS-LLM reaches 2.01% and 2.28%, i.e. $1.95\times$ and $3.21\times$ the reduction of the unexpanded model with the same number of updates.

Expansion does not help when the base model is not capacity-limited.

On Qwen2.5-0.5B the unexpanded model achieves the largest reduction on both datasets (20.43% vs. 14.09% on GSM8K; 25.34% vs. 24.12% on Alpaca). Qwen2.5-0.5B starts from a much lower perplexity on GSM8K (1.91) than SmolLM (6.31), suggesting that its capacity is not the bottleneck on these 30 examples, and the split model has $1.6\times$ more trainable parameters to move with the same learning rate and only 13 optimiser updates. Figure 2 shows the three Qwen conditions tracking each other closely step by step, with the base model slightly lower on GSM8K. We report this regime explicitly: DPS-LLM is a tool for relieving saturation, and a longer schedule or a learning rate tuned for the larger

Table 1: **Fine-tuning benchmark on two open-weight LLMs and two datasets.** All runs use the identical budget of Table 2. *Init PPL* is validation perplexity immediately after splitting and before any update; *Rel. PPL reduction* is $100 (\text{PPL}_{init} - \text{PPL}_{final}) / \text{PPL}_{init}$ (higher is better); *vs. Naive* is the absolute difference in that reduction between DPS-LLM and exact duplication; $\times$ *Base* is the ratio of the reduction to that of the unexpanded model. Single seed (42); see Limitations.

Model	Dataset	Growth method	Params	Init PPL	Final PPL	Rel. PPL red. (%)	vs. Naive / $\times$ Base
Qwen2.5-0.5B	GSM8K	Base (no split)	494.0M	1.91	1.52	20.43	− / 1.00
		Naive split ($C=0$)	807.8M	1.91	1.64	13.83	− / 0.68
		DPS-LLM (ours)	807.8M	1.91	1.64	14.09	+0.26 / 0.69
SmolLM-360M	GSM8K	Base (no split)	361.8M	6.31	6.25	1.03	− / 1.00
		Naive split ($C=0$)	597.8M	6.30	6.18	1.94	− / 1.88
		DPS-LLM (ours)	597.8M	6.31	6.18	2.01	+0.07 / 1.95
Qwen2.5-0.5B	Alpaca	Base (no split)	494.0M	7.29	5.44	25.34	− / 1.00
		Naive split ($C=0$)	807.8M	7.29	5.54	24.03	− / 0.95
		DPS-LLM (ours)	807.8M	7.28	5.53	24.12	+0.09 / 0.95
SmolLM-360M	Alpaca	Base (no split)	361.8M	8.88	8.81	0.71	− / 1.00
		Naive split ($C=0$)	597.8M	8.86	8.70	1.85	− / 2.61
		DPS-LLM (ours)	597.8M	8.86	8.66	2.28	+0.43 / 3.21

Setting	Value
Hardware	Kaggle, 2 $\times$ NVIDIA T4 (16 GB); one model per GPU
Precision	bfloat16 weights and activations
Optimiser	AdamW (Loshchilov and Hutter, 2019), $\beta=(0.9, 0.999)$, weight decay 0.01
Learning rate	2×10^{-5} , constant, no warm-up
Budget	25 sequences (batch 1, accumulation 2 $\Rightarrow$ 13 updates)
Grad. clipping	global norm 1.0
Max seq. length	128 tokens
Trainable	SwiGLU MLP of every layer; rest frozen
Memory	gradient checkpointing (Chen et al., 2016b)
Noise constant	$C=10^{-4}$ (DPS-LLM); $C=0$ (naive)
Random seed	42
Software	PyTorch (Paszke et al., 2019), Transformers (Wolf et al., 2020), Datasets (Lhoest et al., 2021)

Table 2: Fine-tuning hyper-parameters shared by all twelve runs.

model would be needed to assess whether the expanded Qwen eventually overtakes its seed.

5 Conclusion

DPS-LLM grows the SwiGLU feed-forward blocks of a pre-trained LLM in place and without a loss spike, by placing the invariance factor $\frac{1}{2}$ on the down projection and by breaking the symmetry of the duplicated neurons with a polarity-preserving, anti-symmetric perturbation. On two open-weight models and two datasets the expanded model starts exactly where the seed left off, the calculated noise consistently extracts more from the added capac-

ity than exact duplication, and a capacity-limited 360M model adapts two to three times more in relative perplexity than it does unexpanded. The same operator applies to MoE upcycling and LoRA rank growth; evaluating those regimes, longer schedules, and larger seeds are the natural next steps.

Limitations

Scale of the evidence. Each condition was fine-tuned on 30 examples for 25 steps (13 optimiser updates) and evaluated on 10 held-out examples, with a single random seed, because the study was designed to fit within a free Kaggle T4 session. The margins between DPS-LLM and naive duplication (0.07–0.43 percentage points) are consistent in sign across four settings but we have not established confidence intervals; multi-seed runs with larger validation sets are required before the effect size can be stated precisely. **Metric.** We report perplexity only. We do not measure GSM8K answer accuracy or instruction-following quality, and lower perplexity on 10 sequences need not translate into better downstream behaviour. **Data provenance.** The validation examples are drawn from the public training splits of GSM8K and Alpaca-Cleaned; they are unseen during our fine-tuning but may have been seen during the checkpoints’ original training, which likely explains Qwen2.5’s very low initial perplexity on GSM8K. Test-split evaluation would remove this ambiguity. **Model scale and regimes.** Both seeds are below one billion parameters, only width splitting was evaluated, and only the MLP parameters were trained. Depth growth, attention-

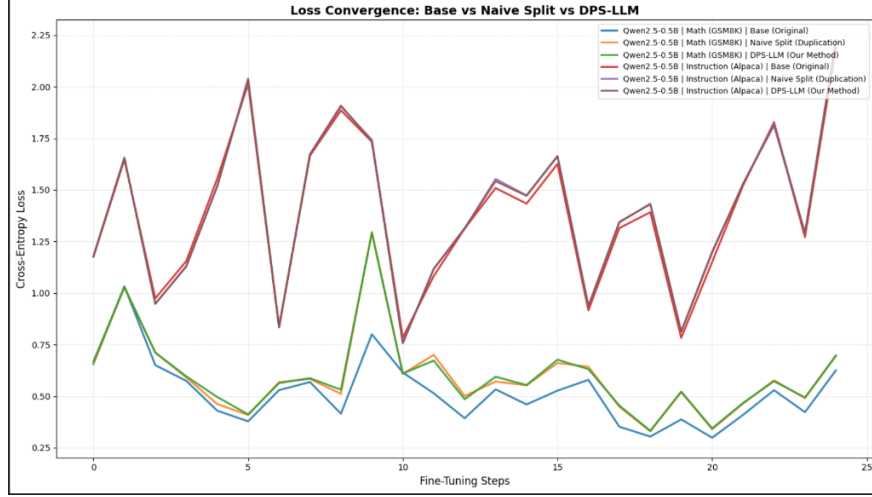


Figure 2: **Per-step training loss on Qwen2.5-0.5B** for the base model, the naive split and DPS-LLM on GSM8K (lower group) and Alpaca (upper group). Each step is one training sequence, so the curves follow the same sequence order and share their shape; the three conditions start from the same loss, illustrating the absence of a split shock, and remain close throughout the 25-step budget.

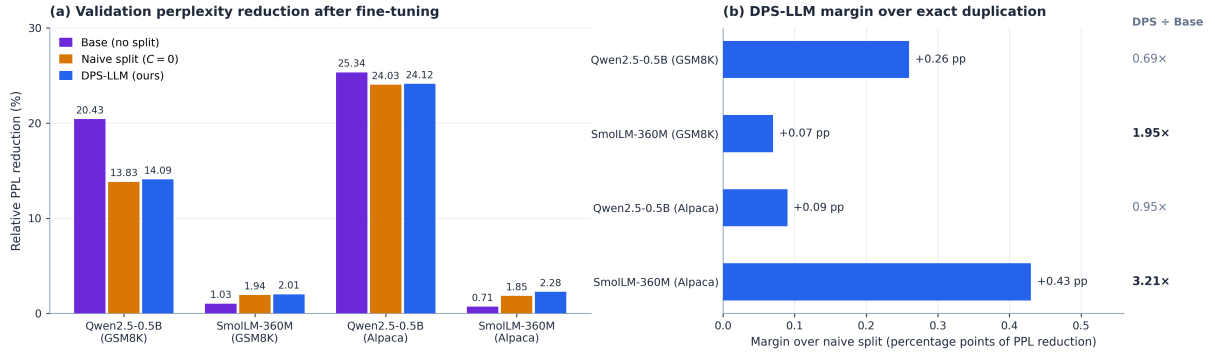


Figure 3: **Benchmark summary.** (a) Relative validation perplexity reduction after fine-tuning for the base model, the naive split and DPS-LLM on each model–dataset pair. (b) Margin of DPS-LLM over exact duplication in percentage points, with the ratio of the DPS-LLM reduction to the base model’s reduction in the right-hand column; DPS-LLM is ahead of duplication in all four settings, and ahead of the unexpanded model only on the capacity-limited SmoLLM-360M.

head splitting, the MoE and LoRA regimes, and full-parameter fine-tuning are untested. **Numerics.** Invariance is exact in real arithmetic; in bfloat16 we observe initial-perplexity deviations of up to 0.02. On the T4 GPU bfloat16 arithmetic is emulated, so absolute timings were not meaningful and are not reported. **Hyper-parameters.** The learning rate and schedule were fixed across conditions and were not tuned for the expanded models, which is the most likely reason the expanded Qwen2.5 trails its seed within the budget.

Ethics Statement

This work modifies the architecture of publicly released language models and uses two public datasets (GSM8K, MIT licence; Alpaca-

Cleaned, CC BY-NC 4.0) and two public checkpoints (Qwen2.5-0.5B-Instruct and SmoLLM-360M-Instruct, both Apache 2.0). Growing a model does not change the biases or safety properties of its seed, and a grown model inherits any limitations of the original checkpoint; practitioners should re-run their safety evaluations after expansion. All experiments ran on a free cloud GPU tier with a small compute footprint. No human subjects or personal data were involved.

References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al.

2025. SmolLM2: When smol goes big – data-centric training of a small language model. <i>arXiv preprint arXiv:2502.02737</i> .	477
Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. 2024. SmolLM: Blazing fast and remarkably powerful. Hugging Face Blog, https://huggingface.co/blog/smollm .	478
Cheng Chen, Yichun Yin, Lifeng Shang, Xin Jiang, Yujia Qin, Fengyu Wang, Zhi Wang, Xiao Chen, Zhiyuan Liu, and Qun Liu. 2022. bert2BERT: Towards reusable pretrained language models. In <i>Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 2134–2148.	479
Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. 2016a. Net2Net: Accelerating learning via knowledge transfer. In <i>International Conference on Learning Representations (ICLR)</i> .	480
Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016b. Training deep nets with sublinear memory cost. <i>arXiv preprint arXiv:1604.06174</i> .	481
Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. <i>arXiv preprint arXiv:2110.14168</i> .	482
Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. <i>Neural Networks</i> , 107:3–11.	483
Utku Evci, Bart van Merriënboer, Thomas Unterthiner, Max Vladymyrov, and Fabian Pedregosa. 2022. GradMax: Growing neural networks using gradient information. In <i>International Conference on Learning Representations (ICLR)</i> .	484
William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. <i>Journal of Machine Learning Research</i> , 23(120):1–39.	485
Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In <i>Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)</i> , pages 249–256.	486
Linyuan Gong, Di He, Zhuohan Li, Tao Qin, Liwei Wang, and Tie-Yan Liu. 2019. Efficient training of BERT by progressively stacking. In <i>Proceedings of the 36th International Conference on Machine Learning (ICML)</i> , pages 2337–2346.	487
Ethan He, Abhinav Khattar, Ryan Prenger, Vijay Korthikanti, Zijie Yan, Tong Liu, Shiqing Fan, Ashwath Aithal, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Upcycling large language models into mixture of experts. <i>arXiv preprint arXiv:2410.07524</i> .	488
Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In <i>Proceedings of the IEEE International Conference on Computer Vision (ICCV)</i> , pages 1026–1034.	489
Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (GELUs). <i>arXiv preprint arXiv:1606.08415</i> .	490
Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. In <i>Advances in Neural Information Processing Systems</i> , volume 35.	491
Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In <i>International Conference on Learning Representations (ICLR)</i> .	492
Muhammad Irfan. 2021. Discriminative splitting of LSTM-RNN deep neural networks for robust speech recognition. Master’s thesis, School of Electrical Engineering and Computer Science (SEECS), National University of Sciences and Technology (NUST), Islamabad, Pakistan.	493
Fred Jelinek, Robert L Mercer, Lalit R Bahl, and James K Baker. 1977. Perplexity—a measure of the difficulty of speech recognition tasks. <i>The Journal of the Acoustical Society of America</i> , 62(S1):S63–S63.	494
Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. <i>arXiv preprint arXiv:2401.04088</i> .	495
Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. <i>arXiv preprint arXiv:2001.08361</i> .	496
Dahyun Kim, Chanjun Park, Sanghoon Kim, Wonsung Lee, Wonho Song, Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Kim, Jihoo Kim, et al. 2024. SOLAR 10.7B: Scaling large language models with simple yet effective depth up-scaling. In <i>Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)</i> , pages 23–35.	497
Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Neil Houlsby, and Mostafa Dehghani. 2023. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In <i>International Conference on Learning Representations (ICLR)</i> .	498
Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim	499

533	Krikun, Noam Shazeer, and Zhifeng Chen. 2021.	Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier	587
534	GShard: Scaling giant models with conditional com-	Martinet, Marie-Anne Lachaux, Timothée Lacroix,	588
535	putation and automatic sharding. In <i>International</i>	Baptiste Rozière, Naman Goyal, Eric Hambro,	589
536	<i>Conference on Learning Representations (ICLR)</i> .	Faisal Azhar, et al. 2023. LLaMA: Open and ef-	590
		ficient foundation language models. <i>arXiv preprint</i>	591
537	Quentin Lhoest, Albert Villanova del Moral, Yacine	<i>arXiv:2302.13971</i> .	592
538	Jernite, Abhishek Thakur, Patrick von Platen, Suraj		
539	Patil, Julien Chaumond, Mariama Drame, Julien Plu,	Mojtaba Valipour, Mehdi Rezagholizadeh, Ivan	593
540	Lewis Tunstall, et al. 2021. Datasets: A community	Kobyzev, and Ali Ghodsi. 2023. DyLoRA:	594
541	library for natural language processing. In <i>Proceed-</i>	Parameter-efficient tuning of pre-trained models us-	595
542	<i>ings of the 2021 Conference on Empirical Methods</i>	ing dynamic search-free low-rank adaptation. In <i>Pro-</i>	596
543	<i>ceedings in Natural Language Processing: System Demon-</i>	<i>ceedings of the 17th Conference of the European</i>	597
544	<i>strations</i> , pages 175–184.	<i>Chapter of the Association for Computational Lin-</i>	598
		<i>guistics (EACL)</i> , pages 3274–3287.	599
545	Qiang Liu, Lemeng Wu, and Dilin Wang. 2019. Split-		
546	ting steepest descent for growing neural architectures.	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob	600
547	In <i>Advances in Neural Information Processing Sys-</i>	Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz	601
548	<i>tems</i> , volume 32.	Kaiser, and Illia Polosukhin. 2017. Attention is all	602
		you need. In <i>Advances in Neural Information Pro-</i>	603
549	Ilya Loshchilov and Frank Hutter. 2019. Decoupled	<i>cessing Systems</i> , volume 30.	604
550	weight decay regularization. In <i>International Confer-</i>		
551	<i>ence on Learning Representations (ICLR)</i> .	Peihao Wang, Rameswar Panda, Lucas Torroba Hen-	605
		nigen, Philip Greengard, Leonid Karlinsky, Roge-	606
552	Adam Paszke, Sam Gross, Francisco Massa, Adam	rio Feris, David Daniel Cox, Zhangyang Wang, and	607
553	Lerer, James Bradbury, Gregory Chanan, Trevor	Yoon Kim. 2023. Learning to grow pretrained mod-	608
554	Killeen, Zeming Lin, Natalia Gimelshein, Luca	els for efficient transformer training. In <i>International</i>	609
555	Antiga, et al. 2019. PyTorch: An imperative style,	<i>Conference on Learning Representations (ICLR)</i> .	610
556	high-performance deep learning library. In <i>Ad-</i>		
557	<i>vances in Neural Information Processing Systems</i> ,	Yite Wang, Jiahao Su, Hanlin Lu, Cong Xie, Tianyi Liu,	611
558	volume 32.	Jianbo Yuan, Haibin Lin, Ruoyu Sun, and Hongxia	612
		Yang. 2024. LEMON: Lossless model expansion. In	613
559	Prajit Ramachandran, Barret Zoph, and Quoc V Le.	<i>International Conference on Learning Representa-</i>	614
560	2017. Searching for activation functions. <i>arXiv</i>	<i>tions (ICLR)</i> .	615
561	<i>preprint arXiv:1710.05941</i> .		
562	Mohammad Samragh, Iman Mirzadeh, Keivan Al-	Tao Wei, Changhu Wang, Yong Rui, and Chang Wen	616
563	izadeh Vahid, Fartash Faghri, Minsik Cho, Moin	Chen. 2016. Network morphism. In <i>Proceedings</i>	617
564	Nabi, Devang Naik, and Mehrdad Farajtabar. 2024.	<i>of the 33rd International Conference on Machine</i>	618
565	Scaling smart: Accelerating large language model	<i>Learning (ICML)</i> , pages 564–572.	619
566	pre-training with small model initialization. <i>arXiv</i>		
567	<i>preprint arXiv:2409.12903</i> .	Thomas Wolf, Lysandre Debut, Victor Sanh, Julien	620
		Chaumond, Clement Delangue, Anthony Moi, Pier-	621
568	Noam Shazeer. 2020. GLU variants improve trans-	ric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz,	622
569	former. <i>arXiv preprint arXiv:2002.05202</i> .	et al. 2020. Transformers: State-of-the-art natural	623
		language processing. In <i>Proceedings of the 2020</i>	624
570	Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz,	<i>Conference on Empirical Methods in Natural Lan-</i>	625
571	Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff	<i>guage Processing: System Demonstrations</i> , pages	626
572	Dean. 2017. Outrageously large neural networks:	38–45.	627
573	The sparsely-gated mixture-of-experts layer. In <i>In-</i>		
574	<i>ternational Conference on Learning Representations</i>	Lemeng Wu, Bo Liu, Peter Stone, and Qiang Liu. 2020.	628
575	<i>(ICLR)</i> .	Firefly neural architecture descent: a general ap-	629
		proach for growing neural networks. In <i>Advances in</i>	630
576	Sheng Shen, Pete Walsh, Kurt Keutzer, Jesse Dodge,	<i>Neural Information Processing Systems</i> , volume 33.	631
577	Matthew Peters, and Iz Beltagy. 2022. Staged train-		
578	ing for transformer language models. In <i>Proceedings</i>	yahma. 2023. Alpaca-cleaned: A cleaned and cu-	632
579	<i>of the 39th International Conference on Machine</i>	rated version of the Stanford Alpaca dataset. Hug-	633
580	<i>Learning (ICML)</i> , pages 19893–19908.	ging Face Datasets, https://huggingface.co/	634
		datasets/yahma/alpaca-cleaned .	635
581	Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann		
582	Dubois, Xuechen Li, Carlos Guestrin, Percy	An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui,	636
583	Liang, and Tatsunori B Hashimoto. 2023. Stan-	Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu,	637
584	ford Alpaca: An instruction-following LLaMA	Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 tech-	638
585	model. GitHub repository, https://github.com/	nical report. <i>arXiv preprint arXiv:2412.15115</i> .	639
586	tatsu-lab/stanford_alpaca .		
		Yiqun Yao, Zheng Zhang, Jing Li, and Yequan Wang.	640
		2024. Masked structural growth for 2x faster lan-	641
		guage model pre-training. In <i>International Confer-</i>	642
		<i>ence on Learning Representations (ICLR)</i> .	643

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations (ICLR)*.

A Additional Growth Regimes

The same rows/columns operator and anti-symmetric noise apply to two further situations. Both are implemented in the released code but were not part of the benchmark in Section 4.1.

Dense-to-MoE upcycling. A dense SwiGLU block is cloned into E experts with independent calculated-noise draws $\Delta^{(e)}$ and combined by a top- k softmax router (Shazeer et al., 2017; Fedus et al., 2022):

$$\text{Expert}_e(x) = \text{SwiGLU}(x; W + \Delta^{(e)}), \quad (8)$$

$$g(x) = \text{Top-}k(\text{softmax}(W_r x)). \quad (9)$$

With $k = E$ and a uniform router the layer is initially function-preserving; the perturbations give each expert a distinct gradient trajectory from the first step, which is the failure mode that sparse upcycling (Komatsuzaki et al., 2023) must otherwise wait for the router to resolve.

LoRA rank doubling. For a LoRA adapter $\Delta W = BA$ with $A \in \mathbb{R}^{r \times d_{in}}$, $B \in \mathbb{R}^{d_{out} \times r}$ (Hu et al., 2022), the rank is grown $r \rightarrow 2r$ by

$$A' = \begin{bmatrix} A \\ A + \Delta_A \end{bmatrix}, \quad B' = [B \quad \Delta_B], \quad (10)$$

where Δ_B is calculated noise on a zero matrix, i.e. near zero. Because the new columns of B are (near) zero, $B'A' = BA + \Delta_B(A + \Delta_A) \approx BA$, so the adapted function is preserved while the optimiser gains r additional rank-one directions.

B Prompt Templates

Each example is rendered as plain text and tokenised with the checkpoint’s own tokeniser; no chat template is applied.

Dataset	Template
GSM8K	Question: {question}\n Answer: {answer}
Alpaca	Instruction: {instruction}\n [Input: {input}\n Response: {output}

The bracketed Input line is emitted only when the example has a non-empty input field.

C Reproducibility

The split operator (`split_huggingface_llm.py`), the synthetic experiment behind Figure 1, and the Kaggle notebook that produced Table 1 are available at <https://github.com/mirfan899/parameter-splitting-llm>. The benchmark notebook runs end-to-end on a free Kaggle session with two T4 GPUs in under an hour and writes the results table and per-step loss histories to disk.